

Statistics

Summary

- `ggplot()` specifies what data to use and what variables will be mapped to where
- inside `ggplot()`, `aes(x = , y = , color = )` specify what variables correspond to what aspects of the plot in general
- layers of plots can be combined using the `+` at the **end** of lines
- use `geom_line()` and `geom_point()` to add lines and points
- sometimes you need to add a `group` element to `aes()` if your plot looks strange
- make sure you are plotting what you think you are by checking the numbers!
- `facet_grid(~variable)` and `facet_wrap(~variable)` can be helpful to quickly split up your plot

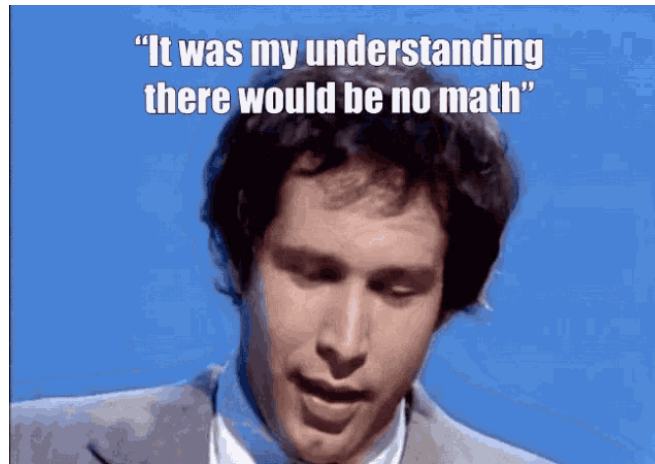
Summary

- the factor class allows us to have a different order from alphanumeric for categorical data
- we can change data to be a factor variable using `mutate()`, `as_factor()` (in the `forcats` package), or `factor()` functions and specifying the levels with the `levels` argument
- `fct_reorder({variable_to_reorder}, {variable_to_order_by})` helps us reorder a variable by the values of another variable
- arranging, tabulating, and plotting the data will reflect the new order

Overview

We will cover how to use R to compute some of basic statistics and fit some basic statistical models.

- Correlation
- T-test
- Linear Regression / Logistic Regression



Overview

We will focus on how to use R software to do these. We will be glossing over the statistical **theory** and “formulas” for these tests. Moreover, we do not claim the data we use for demonstration meet **assumptions** of the methods.

There are plenty of resources online for learning more about these methods.

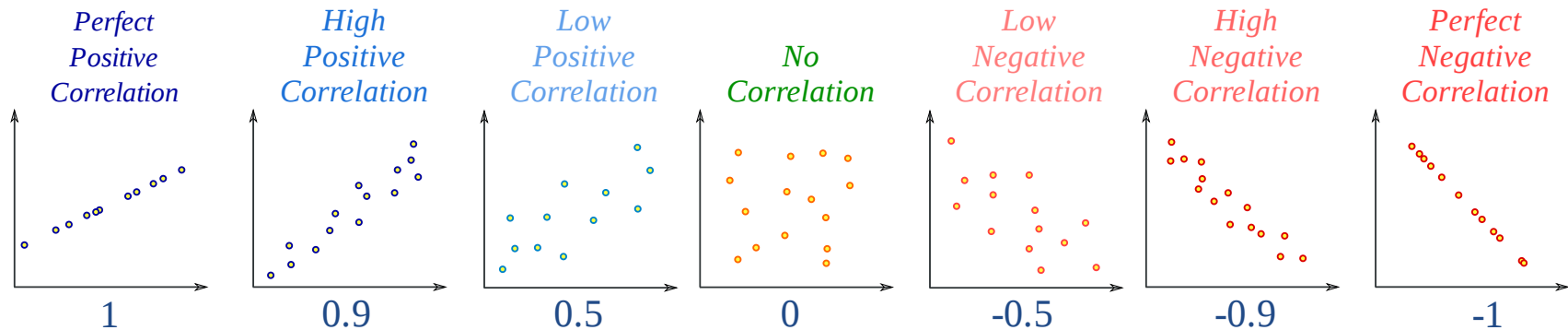
Check out www.opencasestudies.org for deeper dives on some of the concepts covered here and the [resource page](#) for more resources.

Correlation

Correlation

The correlation coefficient is a summary statistic that measures the strength of a linear relationship between two numeric variables.

- The strength of the relationship - based on how well the points form a line
- The direction of the relationship - based on if the points progress upward or downward



[source](#)

See this [case study](#) for more information.

Correlation

Function `cor()` computes correlation in R.

```
cor(x, y = NULL, use = c("everything", "complete.obs"),  
    method = c("pearson", "kendall", "spearman"))
```

- provide two numeric vectors of the same length (arguments `x`, `y`), or
- provide a `data.frame` / `tibble` with numeric columns only
- by default, Pearson correlation coefficient is computed

Correlation test

Function `cor.test()` also computes correlation and tests for association.

```
cor.test(x, y = NULL, alternative=c("two.sided", "less", "greater"),  
        method = c("pearson", "kendall", "spearman"))
```

- provide two numeric vectors of the same length (arguments `x`, `y`), or
- provide a `data.frame` / `tibble` with numeric columns only
- by default, Pearson correlation coefficient is computed
- alternative values:
 - `two.sided` means true correlation coefficient is not equal to zero (default)
 - `greater` means true correlation coefficient is > 0 (positive relationship)
 - `less` means true correlation coefficient is < 0 (negative relationship)

GUT CHECK!

What class of data do you need to calculate a correlation?

A. Character data

B. Factor data

C. Numeric data

Correlation

Let's look at the dataset of yearly CO2 emissions by country.

```
yearly_co2 <-  
  read_csv(file = "https://daseh.org/data/Yearly\_CO2\_Emissions\_1000\_tonnes.csv")
```

Correlation for two vectors

First, we create two vectors.

```
# x and y must be numeric vectors  
y1980 <- yearly_co2 |> pull(`1980`)  
y1985 <- yearly_co2 |> pull(`1985`)
```

Like other functions, if there are NAs, you get NA as the result. But if you specify `use = "complete.obs"`, then it will give you correlation using the non-missing data.

```
cor(y1980, y1985, use = "complete.obs")
```

```
[1] 0.9936257
```

Correlation coefficient calculation and test

```
cor.test(y1980, y1985)
```

Pearson's product-moment correlation

```
data: y1980 and y1985
```

```
t = 114.59, df = 169, p-value < 0.0000000000000000022
```

```
alternative hypothesis: true correlation is not equal to 0
```

```
95 percent confidence interval:
```

```
0.9913844 0.9952853
```

```
sample estimates:
```

```
cor
```

```
0.9936257
```

Broom package

The broom package helps make stats results look tidy

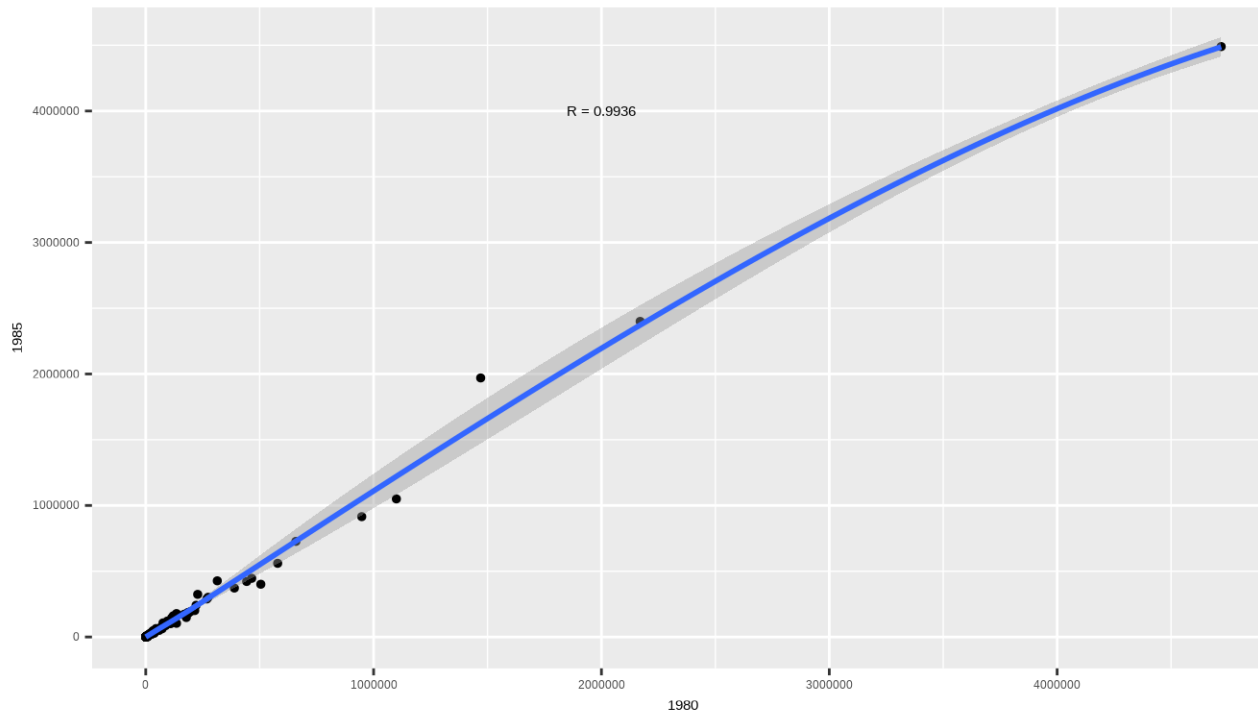
```
library(broom)
cor_result <- tidy(cor.test(y1980, y1985))
glimpse(cor_result)
```

```
Rows: 1  
Columns: 8  
$ estimate      <dbl> 0.9936257  
$ statistic     <dbl> 114.5851  
$ p.value       <dbl> 0.00000000000000000000000000000000000000000000000000000...  
$ parameter     <int> 169  
$ conf.low      <dbl> 0.9913844  
$ conf.high     <dbl> 0.9952853  
$ method        <chr> "Pearson's product-moment correlation"  
$ alternative    <chr> "two.sided"
```

Correlation for two vectors with plot

In plot form... `geom_smooth()` and `annotate()` can look very nice!

```
corr_value <- pull(cor_result, estimate) |> round(digits = 4)
cor_label <- paste0("R = ", corr_value)
yearly_co2 |>
  ggplot(aes(x = `1980`, y = `1985`)) + geom_point(size = 1) + geom_smooth() +
  annotate("text", x = 2000000, y = 4000000, label = cor_label)
```



Correlation for data frame columns

We can compute correlation for all pairs of columns of a data frame / matrix.
This is often called, *“computing a correlation matrix”*.

Columns must be all numeric!

```
co2_subset <- yearly_co2 |>  
  select(c(`1950`, `1980`, `1985`, `2010`))
```

```
head(co2_subset)
```

```
# A tibble: 6 × 4  
  `1950` `1980` `1985` `2010`  
  <dbl> <dbl> <dbl> <dbl>  
1    84.3   1760   3510   8460  
2    297    5170   7880   4600  
3  3790   66500  72800 119000  
4    NA      NA      NA    517  
5   187    5350   4700  29100  
6    NA    143    249    524
```

Correlation for data frame columns

We can compute correlation for all pairs of columns of a data frame / matrix. This is often called, *“computing a correlation matrix”*.

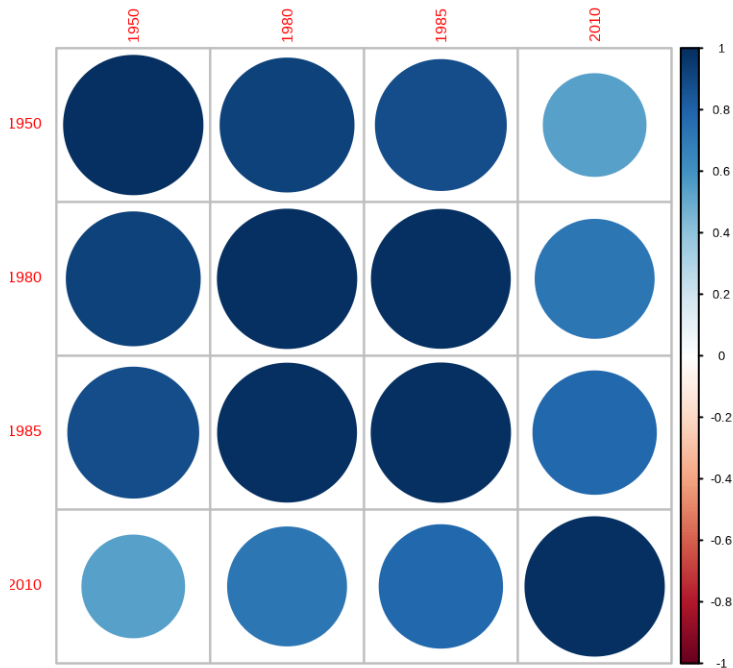
```
cor_mat <- cor(co2_subset, use = "complete.obs")  
cor_mat
```

	1950	1980	1985	2010
1950	1.0000000	0.9228253	0.8818288	0.5415047
1980	0.9228253	1.0000000	0.9935477	0.7270839
1985	0.8818288	0.9935477	1.0000000	0.7827256
2010	0.5415047	0.7270839	0.7827256	1.0000000

Correlation for data frame columns with plot

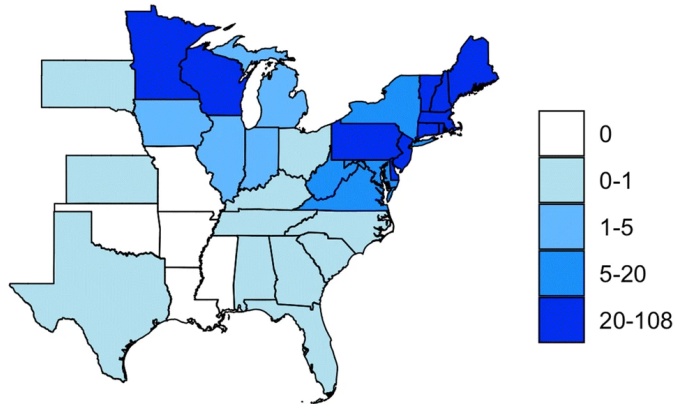
corrplot package can make correlation matrix plots

```
library(corrplot)  
corrplot(cor_mat)
```

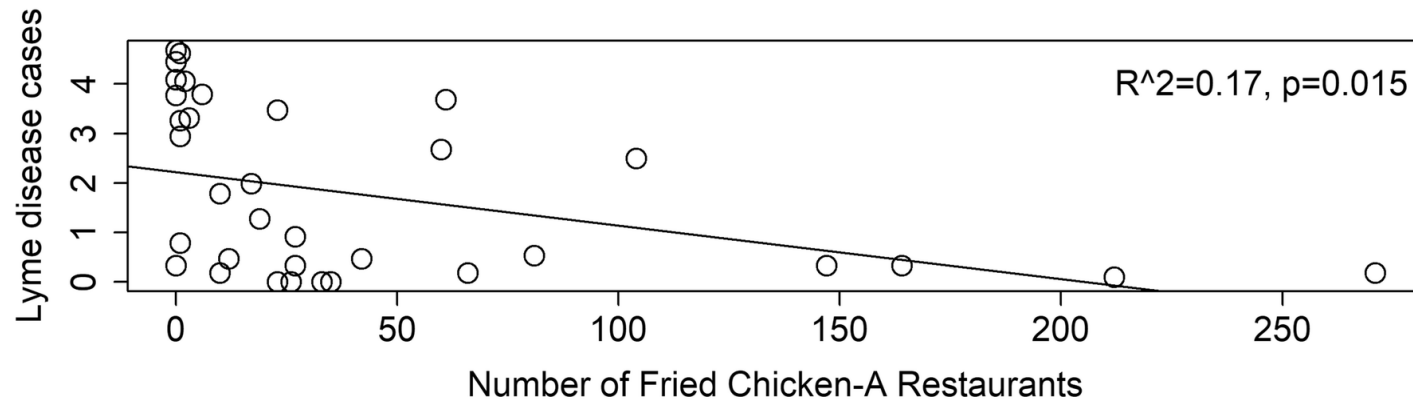
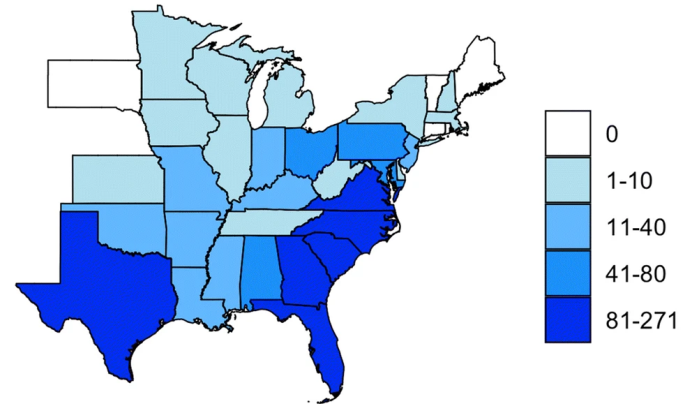


Correlation does not imply causation

Lyme disease incidence



Number of fried chicken restaurants (chain A)



[source](#)

T-test

T-test

The commonly used t-tests are:

- **one-sample t-test** – used to test mean of a variable in one group
- **two-sample t-test** – used to test difference in means of a variable between two groups
 - if the “two groups” are data of the *same* individuals collected at 2 time points, we say it is two-sample paired t-test)

The `t.test()` function does both.

```
t.test(x, y = NULL,  
       alternative = c("two.sided", "less", "greater"),  
       mu = 0, paired = FALSE, var.equal = FALSE,  
       conf.level = 0.95, ...)
```

Running one-sample t-test

It tests the mean of a variable in one group. By default (i.e., without us explicitly specifying values of other arguments):

- tests whether a mean of a variable is equal to 0 (`mu = 0`)
- uses “two sided” alternative (`alternative = "two.sided"`)
- returns result assuming confidence level 0.95 (`conf.level = 0.95`)
- omits **NA** values in data

Let's look at the CO2 emissions data again.

```
t.test(y1980)
```

```
One Sample t-test
```

```
data: y1980
t = 3.3324, df = 170, p-value = 0.001056
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 44745.81 174792.25
sample estimates:
mean of x
 109769
```

Running two-sample t-test

It tests the difference in means of a variable between two groups. By default:

- tests whether difference in means of a variable is equal to 0 (`mu = 0`)
- uses “two sided” alternative (`alternative = "two.sided"`)
- returns result assuming confidence level 0.95 (`conf.level = 0.95`)
- assumes data are not paired (`paired = FALSE`)
- assumes true variance in the two groups is not equal (`var.equal = FALSE`)
- omits NA values in data

Check out this [case study](#) and this [case study](#) for more information.

Running two-sample t-test in R

```
t.test(y1980, y1985)
```

```
Welch Two Sample t-test
```

```
data: y1980 and y1985
```

```
t = -0.090533, df = 341, p-value = 0.9279
```

```
alternative hypothesis: true difference in means is not equal to 0
```

```
95 percent confidence interval:
```

```
-95902.79  87462.97
```

```
sample estimates:
```

```
mean of x mean of y
```

```
109769.0  113988.9
```

T-test: retrieving information from the result with **broom** package

The **broom** package has a `tidy()` function that can organize results into a data frame so that they are easily manipulated (or nicely printed)

```
result <- t.test(y1980, y1985)
result_tidy <- tidy(result)
glimpse(result_tidy)
```

```
Rows: 1
Columns: 10
$ estimate      <dbl> -4219.909
$ estimate1     <dbl> 109769
$ estimate2     <dbl> 113988.9
$ statistic     <dbl> -0.09053303
$ p.value       <dbl> 0.9279168
$ parameter     <dbl> 340.999
$ conf.low      <dbl> -95902.79
$ conf.high     <dbl> 87462.97
$ method        <chr> "Welch Two Sample t-test"
$ alternative    <chr> "two.sided"
```

P-value adjustment

You run an increased risk of Type I errors (a “false positive”) when multiple hypotheses are tested simultaneously.

Use the `p.adjust()` function on a vector of p values. Use `method =` to specify the adjustment method:

```
my_pvalues <- c(0.049, 0.001, 0.31, 0.00001)
p.adjust(my_pvalues, method = "BH") # Benjamini Hochberg
```

```
[1] 0.06533333 0.00200000 0.31000000 0.00004000
```

```
p.adjust(my_pvalues, method = "bonferroni") # multiply by number of tests
```

```
[1] 0.19600 0.00400 1.00000 0.00004
```

```
my_pvalues * 4
```

```
[1] 0.19600 0.00400 1.24000 0.00004
```

See [here](#) for more about multiple testing correction. Bonferroni also often done as p value threshold divided by number of tests (0.05/test number).

Some other statistical tests

- `wilcox.test()` – Wilcoxon signed rank test, Wilcoxon rank sum test
- `shapiro.test()` – Test normality assumptions
- `ks.test()` – Kolmogorov-Smirnov test
- `var.test()` – Fisher's F-Test
- `chisq.test()` – Chi-squared test
- `aov()` – Analysis of Variance (ANOVA)

Summary

- Use `cor()` to calculate correlation between two vectors, `cor.test()` can give more information.
- `corrplot()` is nice for a quick visualization!
- `t.test()` one sample test to test the difference in mean of a single vector from zero (one input)
- `t.test()` two sample test to test the difference in means between two vectors (two inputs)
- `tidy()` in the **broom** package is useful for organizing and saving statistical test output
- Remember to adjust p-values with `p.adjust()` when doing multiple tests on data

Lab Part 1

▮ [Class Website](#)

▮ [Lab](#)

Regression

Linear regression

Linear regression is a method to model the relationship between a response and one or more explanatory variables.

Most commonly used statistical tests are actually specialized regressions, including the two sample t-test, [see here for more](#).

Linear regression notation

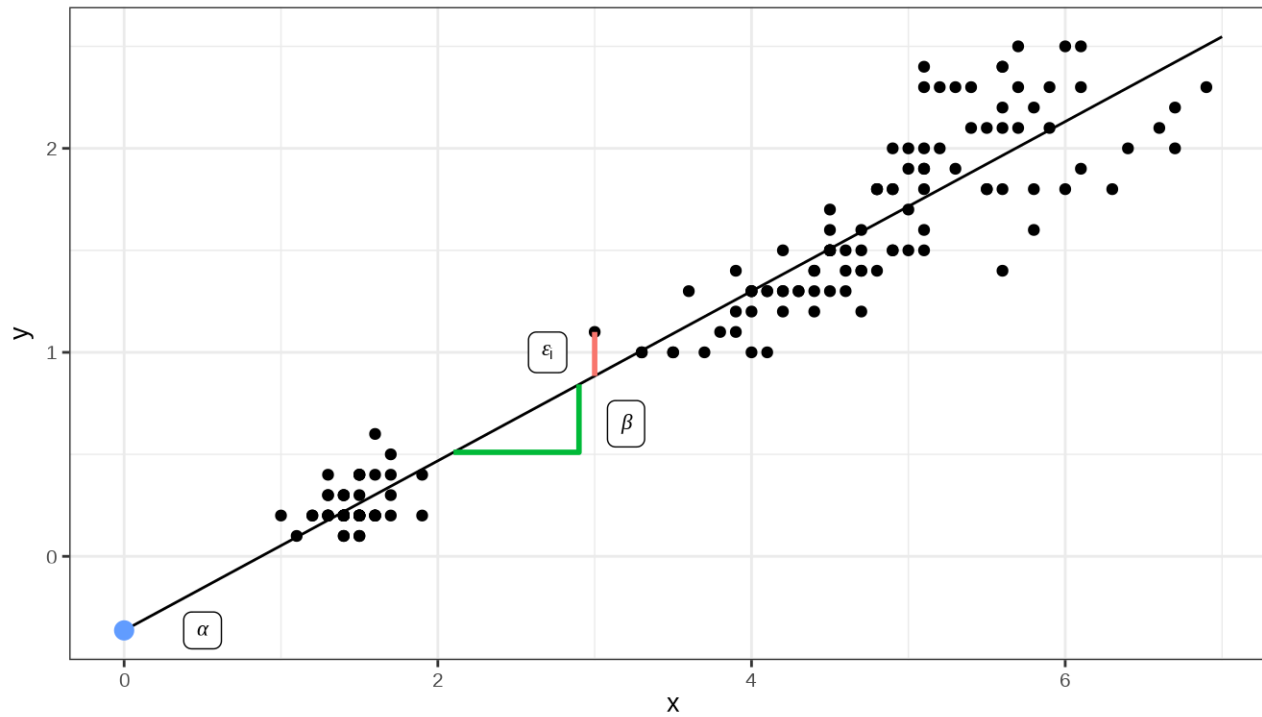
Here is some of the notation, so it is easier to understand the commands/results.

$$y_i = \alpha + \beta x_i + \varepsilon_i$$

where:

- y_i is the outcome for person i
- α is the intercept
- β is the slope (also called a coefficient) - the mean change in y that we would expect for one unit change in x ("rise over run")
- x_i is the predictor for person i
- ε_i is the residual variation for person i

Linear regression



Linear regression

Linear regression is a method to model the relationship between a response and one or more explanatory variables.

We provide a little notation here so some of the commands are easier to put in the proper context.

$$y_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \varepsilon_i$$

where:

- y_i is the outcome for person i
- α is the intercept
- $\beta_1, \beta_2, \beta_3$ are the slopes/coefficients for variables x_{i1}, x_{i2}, x_{i3} - average difference in y for a unit change (or each value) in x while accounting for other variables
- x_{i1}, x_{i2}, x_{i3} are the predictors for person i
- ε_i is the residual variation for person i

See this [case study](#) for more details.

Linear regression fit in R

To fit regression models in R, we use the function `glm()` (Generalized Linear Model).

You may also see `lm()` which is a more limited function that only allows for normally/Gaussian distributed error terms (aka typical linear regressions).

We typically provide two arguments:

- `formula` – model formula written using names of columns in our data
- `data` – our data frame

Linear regression fit in R: model formula

Model formula

$$y_i = \alpha + \beta x_i + \varepsilon_i$$

In R translates to

$$y \sim x$$

Linear regression fit in R: model formula

Model formula

$$y_i = \alpha + \beta x_i + \varepsilon_i$$

In R translates to

$$y \sim x$$

In practice, y and x are replaced with the **names of columns from our data set**.

For example, if we want to fit a regression model where outcome is `income` and predictor is `years_of_education`, our formula would be:

```
income ~ years_of_education
```

Linear regression fit in R: model formula

Model formula

$$y_i = \alpha + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \varepsilon_i$$

In R translates to

$$y \sim x1 + x2 + x3$$

In practice, `y` and `x1`, `x2`, `x3` are replaced with the **names of columns from our data set**.

For example, if we want to fit a regression model where outcome is `income` and predictors are `years_of_education`, `age`, and `location` then our formula would be:

```
income ~ years_of_education + age + location
```

Linear regression example

Let's look variables that might be able to predict the number of "crowded" households.

We'll use a dataset that has socioeconomic measures from CDC. Find out more on <https://daseh.org/data>.

It has already been filtered to include a few counties from Washington State.

Each row represents a census tract/area.

Linear regression example

```
sp_dat <- read_csv(file = "https://daseh.org/data/socioeco_cdc.csv")
```

```
sp_dat
```

```
# A tibble: 927 × 12
```

	county	fips	description	pci	hu	munit	sngpnt	crowd	noveh	f_sngpnt
	<chr>	<dbl>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Clark	53011040101	Census Trac...	27021	1767	6	52	114	110	0
2	Clark	53011040102	Census Trac...	28694	1182	0	28	117	13	0
3	Clark	53011040201	Census Trac...	38606	2430	0	103	25	58	0
4	Clark	53011040202	Census Trac...	32070	1431	0	69	51	7	0
5	Clark	53011040203	Census Trac...	33441	1990	0	52	64	59	0
6	Clark	53011040301	Census Trac...	40783	737	0	37	21	41	0
7	Clark	53011040302	Census Trac...	42532	3041	0	261	19	60	0
8	Clark	53011040403	Census Trac...	43666	1639	0	21	10	19	0
9	Clark	53011040407	Census Trac...	26155	2235	231	138	76	122	0
10	Clark	53011040408	Census Trac...	42397	1269	8	14	28	32	0

```
#   917 more rows
```

```
#   2 more variables: f_crowd <dbl>, f_noveh <dbl>
```

Linear regression: model fitting

For this model, we will use two variables:

- **crowd** - At household level (occupied housing units), more people than rooms
- **hu** - Number of housing units

```
fit <- glm(crowd ~ hu, data = sp_dat)
fit
```

```
Call: glm(formula = crowd ~ hu, data = sp_dat)
```

Coefficients:

(Intercept)	hu
-12.43948	0.03547

Degrees of Freedom: 926 Total (i.e. Null); 925 Residual

Null Deviance: 4223000

Residual Deviance: 3428000 AIC: 10250

Linear regression: model summary

The `summary()` function returns a list that shows us some more detail

```
summary(fit)
```

Call:

```
glm(formula = crowd ~ hu, data = sp_dat)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-12.439483	5.499063	-2.262	0.0239	*
hu	0.035468	0.002422	14.644	<0.000000000000000002	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 3706.214)

Null deviance: 4222997 on 926 degrees of freedom

Residual deviance: 3428248 on 925 degrees of freedom

AIC: 10253

Number of Fisher Scoring iterations: 2

tidy results

The broom package can help us here too!

The estimate is the coefficient or slope.

for every 1 additional housing unit, we see 0.035 more crowded households (~29 housing units to one more crowded household might make more sense!). This relationship appears to be quite strong, with a p value $7.96e-44$!

```
tidy(fit) |> glimpse()
```

```
Rows: 2
```

```
Columns: 5
```

```
$ term      <chr> "(Intercept)", "hu"
```

```
$ estimate  <dbl> -12.43948257, 0.03546794
```

```
$ std.error <dbl> 5.499062635, 0.002422068
```

```
$ statistic <dbl> -2.26211, 14.64366
```

```
$ p.value   <dbl> 0.02392196115632636357895002277018647873774170875549, 0.0000...
```

Linear regression: multiple predictors

Let's try adding another other explanatory variable to our model, average per capita income for each census area (`pci`).

```
fit2 <- glm(crowd ~ hu + pci, data = sp_dat)
summary(fit2)
```

Call:

```
glm(formula = crowd ~ hu + pci, data = sp_dat)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	21.8387846	6.3550308	3.436	0.000616	***
hu	0.0411363	0.0023864	17.238	< 0.00000000000000002	***
pci	-0.0011459	0.0001198	-9.566	< 0.00000000000000002	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 3375.923)

Null deviance: 4222997 on 926 degrees of freedom
Residual deviance: 3119353 on 924 degrees of freedom
AIC: 10167

Number of Fisher Scoring iterations: 2

Linear regression: multiple predictors

Can also use `tidy` and `glimpse` to see the output nicely.

```
fit2 |>  
  tidy() |>  
  glimpse()
```

Rows: 3

Columns: 5

```
$ term      <chr> "(Intercept)", "hu", "pci"  
$ estimate  <dbl> 21.838784553, 0.041136308, -0.001145853  
$ std.error <dbl> 6.3550308287, 0.0023863704, 0.0001197898  
$ statistic <dbl> 3.436456, 17.238023, -9.565525  
$ p.value   <dbl> 0.0006156467274999594379431000490399128466378897428512573242...
```

Linear regression: factors

Factors get special treatment in regression models - lowest level of the factor is the comparison group, and all other factors are **relative** to its values.

Let's add the county (`county`) as a factor into our model. We'll need to convert it to a factor first.

```
sp_dat <- sp_dat |> mutate(county = factor(county))
```

Linear regression: factors

The comparison group that is not listed is treated as intercept. All other estimates are relative to the intercept.

```
fit3 <- glm(crowd ~ hu + pci + county, data = sp_dat)
summary(fit3)
```

```
Call:
glm(formula = crowd ~ hu + pci + county, data = sp_dat)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	42.4231040	7.4265782	5.712	0.0000000150	***
hu	0.0393317	0.0022483	17.494	< 0.00000000000000002	***
pci	-0.0018146	0.0001248	-14.542	< 0.00000000000000002	***
countyKing	35.4770337	6.3140724	5.619	0.0000000255	***
countyPierce	-9.9662254	6.7432134	-1.478	0.140	
countySnohomish	5.4030130	6.9562416	0.777	0.438	
countySpokane	-35.1309611	7.5396924	-4.659	0.0000036378	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 2921.148)

Null deviance: 4222997 on 926 degrees of freedom
Residual deviance: 2687457 on 920 degrees of freedom
AIC: 10037

Number of Fisher Scoring iterations: 2

Linear regression: factors

Maybe we want to use King County as our reference. We can relevel the factor.

The counties are relative to the level that is not listed.

```
sp_dat <-  
  sp_dat |>  
  mutate(county = factor(county,  
    levels = c("King", "Clark", "Pierce", "Snohomish", "Spokane")  
  ))
```

```
fit4 <- glm(crowd ~ hu + pci + county, data = sp_dat)  
summary(fit4)
```

Call:

```
glm(formula = crowd ~ hu + pci + county, data = sp_dat)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	77.9001378	7.7746490	10.020	< 0.00000000000000002	***
hu	0.0393317	0.0022483	17.494	< 0.00000000000000002	***
pci	-0.0018146	0.0001248	-14.542	< 0.00000000000000002	***
countyClark	-35.4770337	6.3140724	-5.619	0.0000000255	***
countyPierce	-45.4432591	5.3237881	-8.536	< 0.00000000000000002	***
countySnohomish	-30.0740208	5.3714389	-5.599	0.0000000285	***
countySpokane	-70.6079948	6.4062707	-11.022	< 0.00000000000000002	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

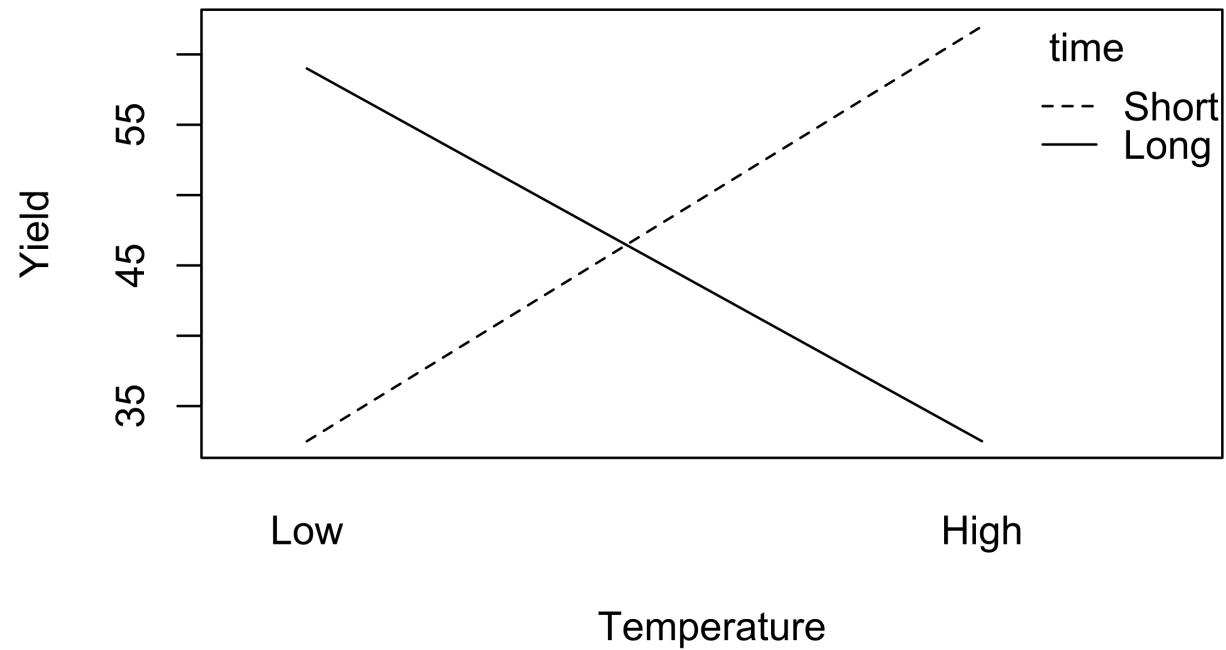
(Dispersion parameter for gaussian family taken to be 2921.148)

Null deviance: 4222997 on 926 degrees of freedom
Residual deviance: 2687457 on 920 degrees of freedom
AIC: 10037

Number of Fisher Scoring iterations: 2

Linear regression: interactions

Interaction plot for cookie baking



[source](#)

Linear regression: interactions

You can also specify interactions between variables in a formula with *. This allows for not only the intercepts between factors to differ, but also the slopes with regard to the interacting variable.

```
fit5 <- glm(crowd ~ hu + pci * county, data = sp_dat)
summary(fit5)
```

Call:

```
glm(formula = crowd ~ hu + pci * county, data = sp_dat)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	73.4590707	8.3703790	8.776	< 0.00000000000000002	***
hu	0.0389502	0.0022620	17.220	< 0.00000000000000002	***
pci	-0.0017060	0.0001442	-11.827	< 0.00000000000000002	***
countyClark	-1.9469215	23.2496384	-0.084	0.9333	
countyPierce	-16.1835554	16.0960723	-1.005	0.3150	
countySnohomish	-17.5168167	18.8785553	-0.928	0.3537	
countySpokane	-86.9928211	19.3206605	-4.503	0.00000758	***
pci:countyClark	-0.0009480	0.0006469	-1.465	0.1431	
pci:countyPierce	-0.0008377	0.0004379	-1.913	0.0561	.
pci:countySnohomish	-0.0003007	0.0004626	-0.650	0.5158	
pci:countySpokane	0.0006271	0.0005971	1.050	0.2939	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 2910.925)

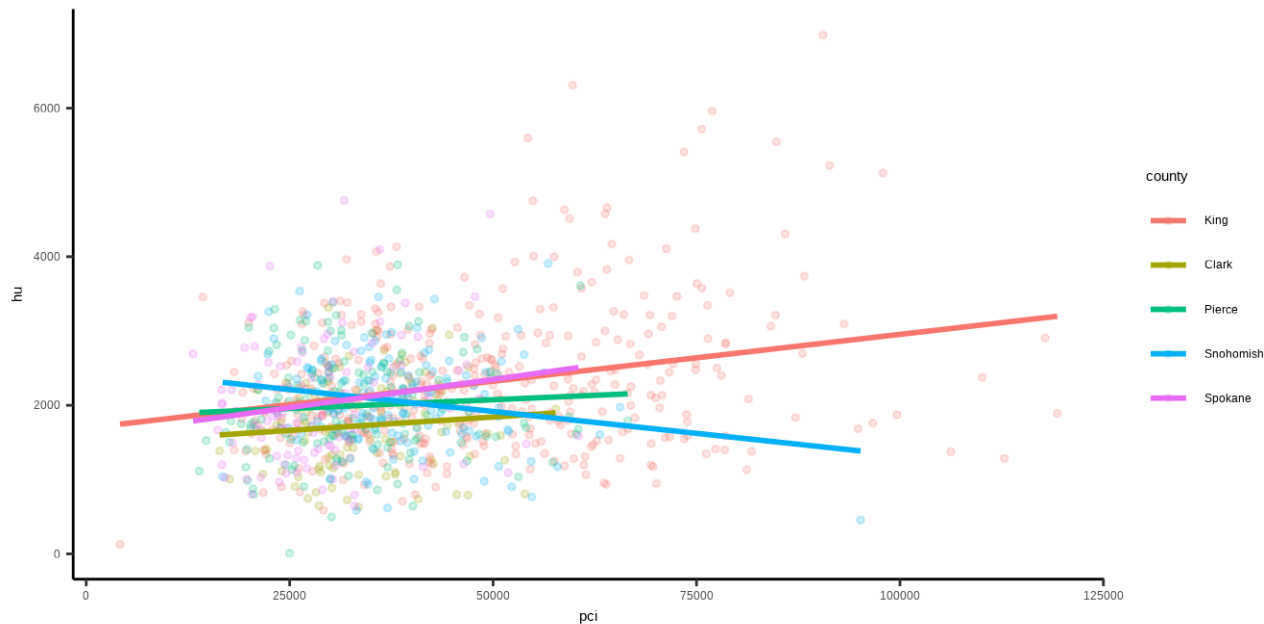
Null deviance: 4222997 on 926 degrees of freedom
Residual deviance: 2666408 on 916 degrees of freedom
AIC: 10038

Number of Fisher Scoring iterations: 2

Linear regression: interactions

By default, `ggplot` with a factor added as a color will look include the interaction term. Notice the different intercept and slope of the lines.

```
ggplot(sp_dat, aes(x = pci, y = hu, color = county)) +  
  geom_point(size = 1, alpha = 0.2) +  
  geom_smooth(method = "glm", se = FALSE) +  
  theme_classic()
```



Generalized linear models (GLMs)

Generalized linear models (GLMs) allow for fitting regressions for non-continuous/normal outcomes. Examples include: logistic regression, Poisson regression.

Add the `family` argument – a description of the error distribution and link function to be used in the model. These include:

- `binomial(link = "logit")` - outcome is binary
- `poisson(link = "log")` - outcome is count or rate
- others

Very important to use the right test!

See this [case study](#) for more information.

See `?family` documentation for details of family functions.

Logistic regression

Let's look at a logistic regression example. We'll use the `sp_dat` dataset again with a different variable.

- **f_crowd** - Flag for the percentage of crowded households is in the 90th percentile (1 = yes, 0 = no)

There are 36 census tracts in the 90th percentile for crowded households.

```
sp_dat |> count(f_crowd)
```

```
# A tibble: 2 × 2
```

	f_crowd	n
	<dbl>	<int>
1	0	891
2	1	36

Logistic regression

Let's explore how hu, pci, and county might predict f_crowd.

```
# General format
glm(y ~ x, data = DATASET_NAME, family = binomial(link = "logit"))

binom_fit <- glm(f_crowd ~ hu + pci + county,
                 data = sp_dat, family = binomial(link = "logit"))
summary(binom_fit)

Call:
glm(formula = f_crowd ~ hu + pci + county, family = binomial(link = "logit"),
    data = sp_dat)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	3.46642282	0.97204634	3.566	0.000362	***
hu	0.00025822	0.00029372	0.879	0.379314	
pci	-0.00019389	0.00003096	-6.263	0.000000000378	***
countyClark	-2.22310417	0.77710614	-2.861	0.004226	**
countyPierce	-2.49172286	0.59240684	-4.206	0.000025981444	***
countySnohomish	-2.13812813	0.76448803	-2.797	0.005161	**
countySpokane	-3.91697302	1.06411805	-3.681	0.000232	***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 304.47 on 926 degrees of freedom
Residual deviance: 199.62 on 920 degrees of freedom
AIC: 213.62

Number of Fisher Scoring iterations: 8

Logistic Regression

See this [case study](#) for more information.

Odds ratios

An odds ratio (OR) is a measure of association between an exposure and an outcome. The OR represents the odds that an outcome will occur given a particular exposure, compared to the odds of the outcome occurring in the absence of that exposure.

Check out [this paper](#).

Use `oddsratio(x, y)` from the `epitools()` package to calculate odds ratios.

Odds ratios

Let's see if a high prevalence of no vehicle homes can predict a high prevalence of crowded homes.

- **f_noveh** - Flag for the percentage of households with no vehicles is in the 90th percentile (1 = yes, 0 = no)
- **f_crowd** - Flag for the percentage of crowded households is in the 90th percentile (1 = yes, 0 = no)

Odds ratios

In this case, we're calculating the odds ratio for census areas, indicating whether a prevalence of no vehicle households is associated with more crowded households.

```
library(epitools)
```

```
response <- sp_dat %>% pull(f_crowd)  
predictor <- sp_dat %>% pull(f_noveh)
```

Odds ratios

The Odds Ratio is 3.33.

When the predictor is 1 (aka the census area has a lot of no vehicle households), the odds of the response (prevalence of crowded homes) are 3.33 times greater than when it is 0 (not a lot of no vehicle households).

```
$data
```

	Outcome		
Predictor	0	1	Total
0	849	31	880
1	42	5	47
Total	891	36	927

```
$measure
```

	odds ratio with 95% C.I.		
Predictor	estimate	lower	upper
0	1.000000	NA	NA
1	3.331243	1.074548	8.385917

```
$p.value
```

	two-sided		
Predictor	midp.exact	fisher.exact	chi.square
0	NA	NA	NA
1	0.03856847	0.03106555	0.01389049

```
$correction
```

```
[1] FALSE
```

```
attr(,"method")
```

```
[1] "median-unbiased estimate & mid-p exact CI"
```

Final note

Some final notes:

- Researcher's responsibility to **understand the statistical method** they use – underlying assumptions, correct interpretation of method results
- Researcher's responsibility to **understand the R software** they use – meaning of function's arguments and meaning of function's output elements

Summary

- `glm()` fits regression models:
 - Use the `formula` = argument to specify the model (e.g., $y \sim x$ or $y \sim x1 + x2$ using column names)
 - Use `data` = to indicate the dataset
 - Use `family` = to do a other regressions like logistic, Poisson and more
 - `summary()` gives useful statistics
- `oddsratio()` from the `epitools` package can calculate odds ratios (outside of logistic regression - which allows more than one explanatory variable)
- this is just the tip of the iceberg!

Resources (also on the [website!](#))

For more check out:

- [this chapter](#) on modeling in this tidyverse book
- [this chart on when to do what test](#)
- opencasestudies.org

Content for similar topics as this course can also be found on Leanpub.

Lab Part 2

- ▮ [Class Website](#)
- ▮ [Lab](#)
- ▮ [Day 8 Cheatsheet](#)



Image by [Gerd Altmann](#) from [Pixabay](#)

Extra Slides

Model Selection

Check out the `leaps` package and other code snippets here: <https://r-statistics.co/Model-Selection-in-R.html>

More tests!

Wilcoxon Test

The Wilcoxon test is a good alternative to the t-test when the normal distribution of the differences between paired individuals cannot be assumed.

```
wilcox.test(x, y, ...)
```

- Like t-test, provide one or two vectors (x, y)
- Choose from `alternative = c("two.sided", "less", "greater")`
- Use `paired = TRUE` for paired values (e.g., before and after)

Shapiro Test

Can tell you if a vector is normally distributed.

```
shapiro.test(x)
```

The smaller the p-value, the more likely the data violates normality assumptions.

Kolmogorov-Smirnov test

Can tell you if two groups come from different distributions.

```
ks.test()
```

The smaller the p-value, the more likely the data are from different distributions.

Fisher's F-Test

Performs an F test to compare the variances of two samples from normal populations.

```
var.test()
```

Chi-squared test

For categorical data/ratios, can tell you if observations match expected values or if two categorical variables are independent.

```
chisq.test()
```


Analysis of Variance (ANOVA)

For balanced designs, determine if multiple variables influence a dependent variable. “Within versus among group variance”.

`aov( )`

More on Linear regression: factors

You can view estimates for the comparison group by removing the intercept in the GLM formula

$y \sim x - 1$

Caveat is that the p-values change, and interpretation is often confusing.

```
fit_force_intercept <-  
  glm(crowd ~ pci + sngpnt + county - 1, data = sp_dat)  
summary(fit_force_intercept)
```

```
Call:  
glm(formula = crowd ~ pci + sngpnt + county - 1, data = sp_dat)
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
pci	-0.0007588	0.0001463	-5.188	0.00000026132	***
sngpnt	0.2379315	0.0215775	11.027	< 0.0000000000000002	***
countyKing	84.8722295	9.2681555	9.157	< 0.0000000000000002	***
countyClark	44.0054938	8.7528775	5.028	0.00000059706	***
countyPierce	36.2601486	8.4082758	4.312	0.00001789033	***
countySnohomish	52.3318984	8.9457720	5.850	0.00000000683	***
countySpokane	15.4316228	8.8378663	1.746	0.0811	.

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 3438.458)

```
Null deviance: 7852783 on 927 degrees of freedom  
Residual deviance: 3163382 on 920 degrees of freedom  
AIC: 10188
```

Number of Fisher Scoring iterations: 2